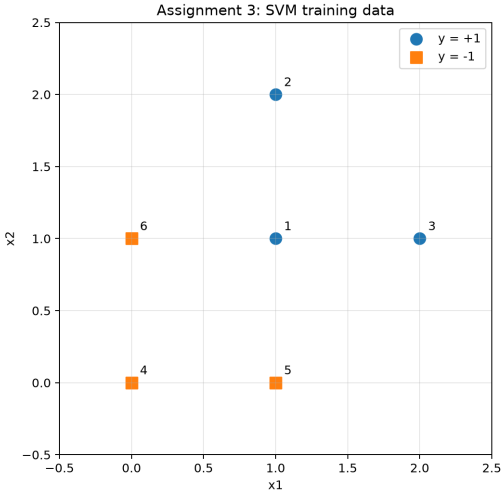
CSE 242 Assignment 3

Question 1

(a) From the plot, the two classes can be separated by a diagonal line. I think the easiest way to see this is to add $x_1 + x_2$ for each point. The positive points (1, 2, 3) get 2, 3, 3, and the negative points (4, 5, 6) get 0, 1, 1, so the gap between the classes is between 1 and 2.

The support vectors are **points 1, 5, and 6**. Point 1 is the closest positive point to the gap, and points 5 and 6 are the closest negative points. Points 2, 3, and 4 are farther away, so they do not decide where the line goes.



Training data

(b) For support vectors, $y(w \cdot x + b) = 1$, so I get these equations:

point 1: $w_1 + w_2 + b = 1$
point 5: $w_1 + b = -1$
point 6: $w_2 + b = -1$

The first equation minus the second gives $w_2 = 2$. The first minus the third gives $w_1 = 2$. Putting this into the second equation gives $b = -3$.

So the line is $\mathbf{2x_1 + 2x_2 - 3 = 0}$, with $w = (2, 2)$ and $b = -3$. It is the same as $x_1 + x_2 = 1.5$.

To check, I plugged in the other points. Point 2 gives $2 + 4 - 3 = 3$, point 3 gives $4 + 2 - 3 = 3$, and point 4 gives -3, so $y(w \cdot x + b) = 3$ for all of them. This is at least 1, so the constraints are satisfied.

Geometric margin (distance from the line to the closest points):

$$\text{margin} = 1 / ||w|| = 1 / \sqrt{(2^2 + 2^2)} = 1 / (2\sqrt{2}) = \sqrt{2} / 4 \approx \mathbf{0.354}$$

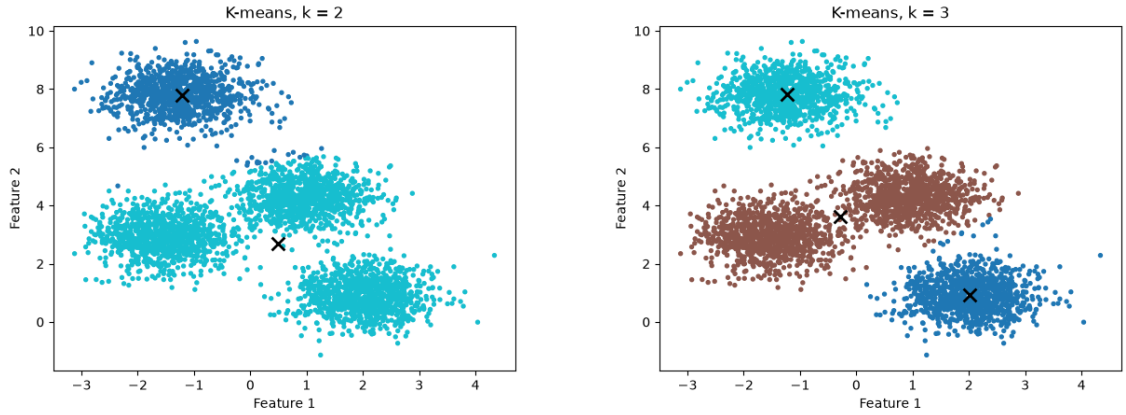
The full width between the two supporting lines would be 2 / ||w|| ≈ 0.707, but this is a separate number. I am reporting 0.354 as the margin.

Question 2

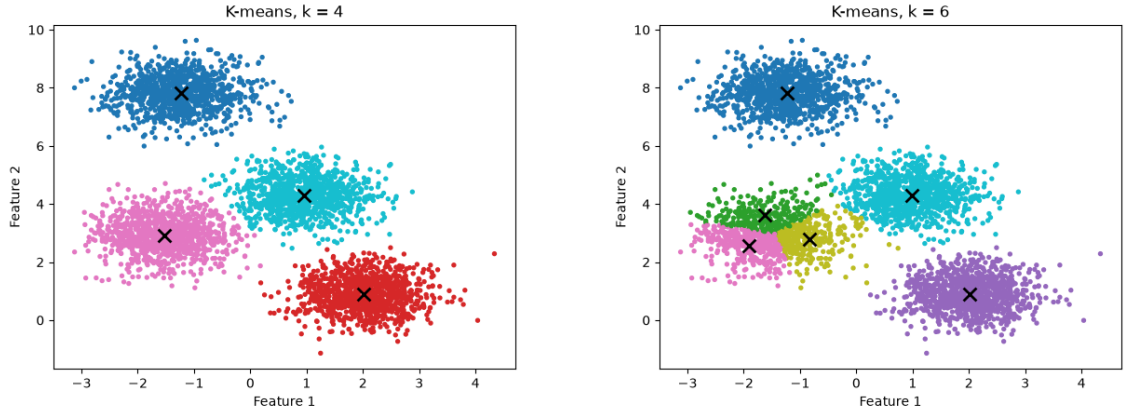
(a) In update_assignments, I calculate the squared distance from each point to each centroid and pick the smallest. I did not take the square root because the closest centroid is the same either way. If there is a tie, np.argmin picks the smaller cluster number.

In update_centroids, each centroid becomes the mean of the points assigned to it. If a cluster is empty, it keeps its previous centroid. None of the clusters were empty in my final results.

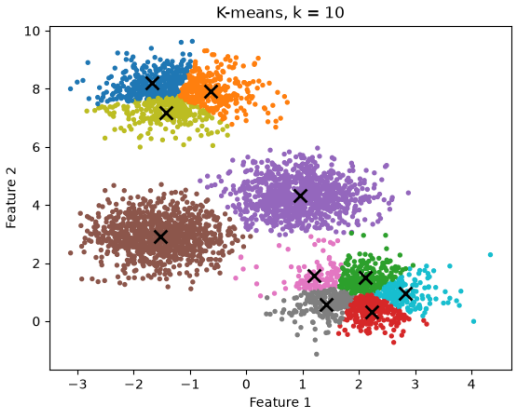
(b) I ran k = 2, 3, 4, 6, 10 with seed 0 and 100 iterations and did not use the true labels. The black x marks are the final centroids.



k = 2 and k = 3



k = 4 and k = 6



k = 10

When k is small, there are fewer centroids than real groups, so k-means has to put groups together. With k = 2, the top group is one cluster and the other three are mostly in the other. With k = 3, the lower-left and middle groups are combined. k = 4 looks the best because each cluster looks like one of the four blobs, and each has about 1,000 points. I only checked this by looking at the plot, not with the true labels.

With k = 6 and k = 10, some blobs get cut into pieces, but not evenly. With k = 6, only the lower-left blob is cut (into 3 pieces). With k = 10, the top blob is cut into 3 and the lower-right blob into 5, while the other two stay mostly whole. I am not completely sure why it happens this way, but my guess is that it depends on where the random starting centroids land. I did not try other seeds, so this could look different with another seed. It seems like a bigger k does not really find more real groups. It just cuts the same data into smaller pieces.

Question 3

I generated one training set and one test set with seed 0 and kept them the same for all 20 trials. In each trial, I flipped positive labels with probability 0.35 and negative labels with probability 0.20. Each label only gets one random draw, so it cannot flip twice. The test labels stay clean. The SVM is RBF with C = 1 and gamma = 0.01.

For K-NN repair, I used NearestNeighbors from sklearn. I asked for 6 neighbors and skipped the first one because that is the point itself, which leaves the 5 nearest other points. The new label is the majority of those 5 noisy labels, and all votes use the same original noisy labels. If distances tie, I just use the order sklearn returns, but I do not expect ties with this kind of data. The repair only uses training features and noisy labels. Both methods use the same noisy labels in each trial.

	Mean test accuracy
Baseline (noisy labels)	0.8280
K-NN repair	0.8644
Difference (K-NN − baseline)	0.0364

The per-trial results are in results/noise_comparison.csv.

K-NN repair helped in my experiment. The mean accuracy went up by about 0.036, and K-NN was higher in all 20 trials. I think this is because the flips are random, so a point with a flipped label usually still has neighbors with mostly correct labels, and the vote can fix it. This seems to give the SVM cleaner data to train on.

I do not think it will always help, though. Where the two classes overlap, a point's neighbors could be mostly from the other class, so a correct label could get changed to a wrong one. Positive labels also flip more often than negative ones (0.35 vs 0.20), and I am not sure how much that affected the votes. I also only used one training and test set, so I cannot say the result would be the same with different data.